

Assignment 1 – Resilient State Machine Replication: Paxos & PBFT

Course: Fundamentals of Distributed Systems

Name: Megha Pandey

Roll Number: G25AI1028

1. Introduction

Distributed systems must continue functioning correctly despite failures occurring in individual nodes or communication links. Modern distributed applications require both crash fault tolerance and Byzantine fault tolerance to ensure reliability, consistency, and security.

This project implements a resilient distributed transaction ledger consisting of five nodes operating in two modes:

1. Mode A – Crash Fault Tolerance

- Leader Election
- Paxos Consensus

2. Mode B – Byzantine Fault Tolerance

- Practical Byzantine Fault Tolerance (PBFT)
- Cryptographic Message Verification

The system is containerized using Docker and Docker Compose. A client continuously submits transactions while network failures and node crashes are simulated using a chaos testing script.

2. System Architecture

The distributed system consists of:

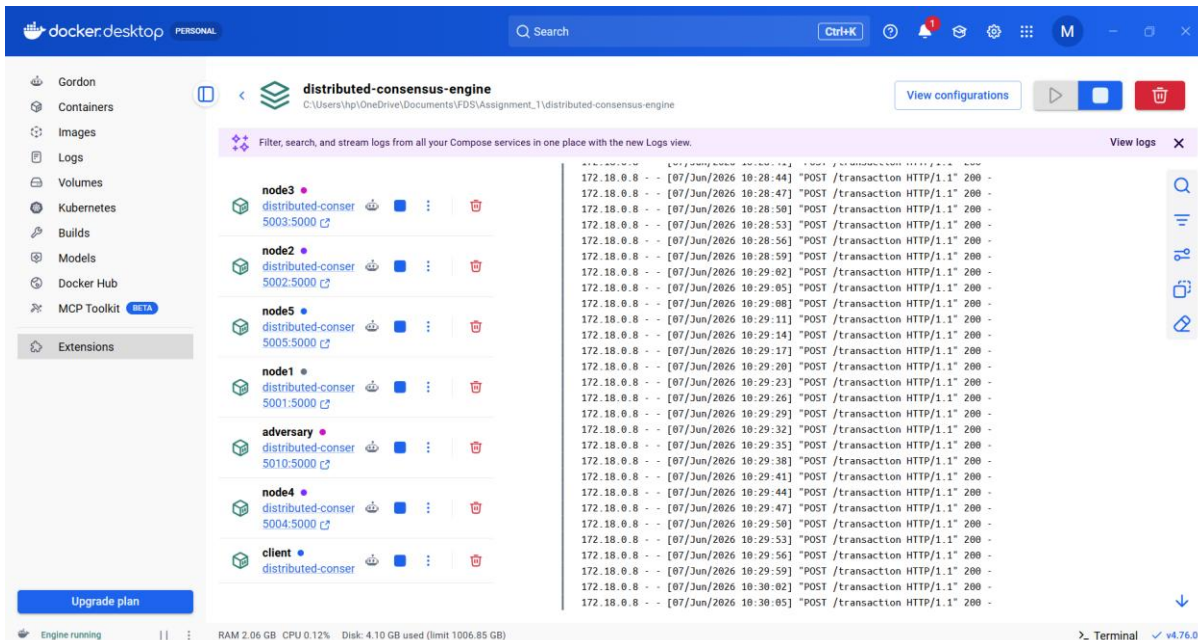
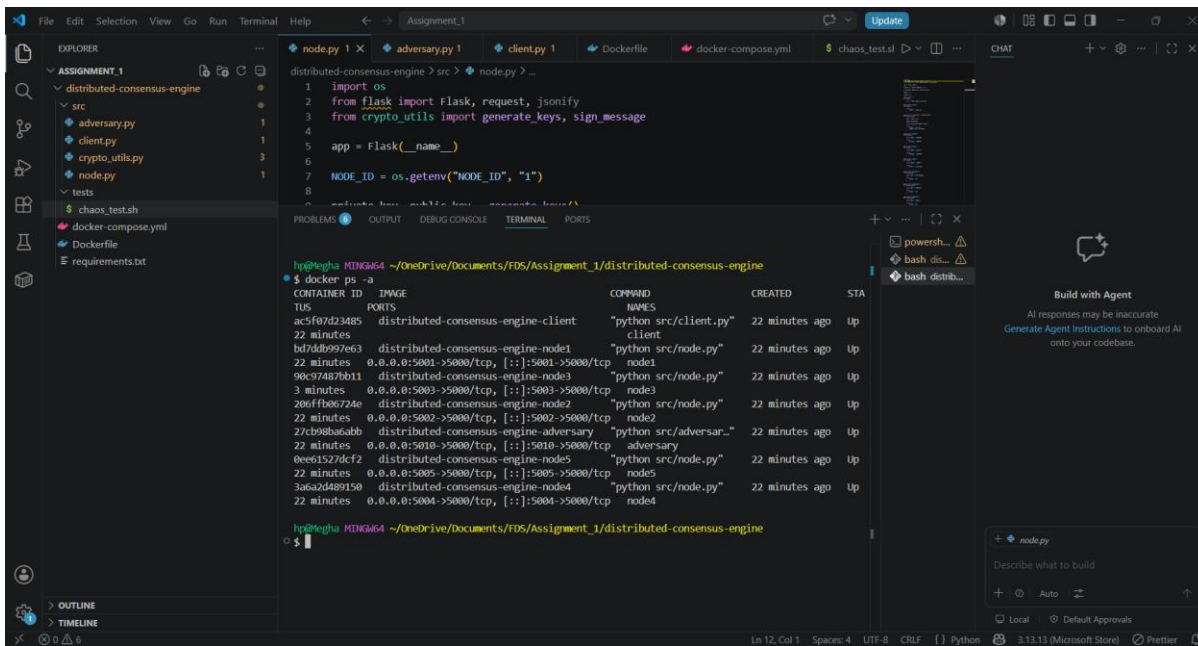
- 5 Consensus Nodes
- 1 Client Node
- 1 Byzantine Adversary Node
- Docker Network for Communication

Architecture Flow:

Client → Leader Node → Consensus Protocol → Distributed Ledger

Each node maintains a local transaction ledger. Consensus protocols ensure all non-faulty nodes agree on the same transaction ordering before committing data.

Caption: Docker Compose Deployment Showing All Running Containers



3. Process Coordination and Leader Election

The system uses a heartbeat-based leader monitoring mechanism.

Working

1. One node acts as the Leader.
2. The Leader periodically sends heartbeat messages.
3. Followers monitor heartbeat intervals.
4. If the Leader becomes unavailable, followers detect timeout.
5. A new Leader is elected automatically.

The elected Leader performs two important roles:

- Paxos Proposer (Mode A)
- PBFT Primary (Mode B)

This design prevents split-brain conditions by ensuring a single active coordinator exists at any time.

State Transition

Follower → Candidate → Leader

Advantages

- Automatic recovery from node failures
- Stable coordination
- Improved fault tolerance

Caption: Leader Election Endpoint Response

```
distributed-consensus-engine > src > node.py > ...
1 import os
2 from flask import Flask, request, jsonify
3 from crypto_utils import generate_keys, sign_message
4
5 app = Flask(__name__)
6
7 NODE_ID = os.getenv("NODE_ID", "1")
8
9 @app.route("/leader", methods=["GET"])
10 def get_leader():
11     return jsonify({"leader": "1"})
12
13 if __name__ == "__main__":
14     app.run(host="0.0.0.0", port=5000)
```

```
hp@hegma MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS
TUS           distributed-consensus-engine-client  "python src/client.py"   22 minutes ago Up
22 minutes    bd7dd997e63   distributed-consensus-engine-node1  "python src/node.py"    22 minutes ago Up
22 minutes    90c97487bb11  distributed-consensus-engine-node3   "python src/node.py"    22 minutes ago Up
3 minutes     0.0.0.0:5001->5000/tcp, [::]:5001->5000/tcp  node3
206ffb06724e   distributed-consensus-engine-node2   "python src/node.py"    22 minutes ago Up
22 minutes    0.0.0.0:5002->5000/tcp, [::]:5002->5000/tcp  node2
27c98ba6abb    distributed-consensus-engine-adversary  "python src/adversar..." 22 minutes ago Up
22 minutes    0.0.0.0:5010->5000/tcp, [::]:5010->5000/tcp  adversary
0ee61527dcf2   distributed-consensus-engine-node5    "python src/node.py"    22 minutes ago Up
22 minutes    0.0.0.0:5005->5000/tcp, [::]:5005->5000/tcp  nodes
3a6a2d489190   distributed-consensus-engine-node4    "python src/node.py"    22 minutes ago Up
22 minutes    0.0.0.0:5004->5000/tcp, [::]:5004->5000/tcp  node4

hp@hegma MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/leader
{"leader":1}
```

4. Paxos Implementation (Crash Fault Tolerance)

Mode A implements Paxos to tolerate crash failures.

The protocol consists of four phases:

4.1 Prepare Phase

The Leader generates a proposal number and sends a PREPARE request to all acceptors.

Purpose:

- Establish proposal ownership
- Prevent conflicting proposals

4.2 Promise Phase

Acceptors respond with PROMISE messages.

Purpose:

- Indicate willingness to accept the proposal
- Reject outdated proposal numbers

4.3 Accept Phase

The Leader sends an ACCEPT request containing the transaction value.

Purpose:

- Propose a transaction for agreement

4.4 Accepted Phase

Acceptors return ACCEPTED responses.

Once a majority is achieved, the transaction is committed to the ledger.

Paxos Workflow

Prepare → Promise → Accept → Accepted → Commit

Fault Tolerance

The system can tolerate up to two simultaneous crash failures while still maintaining consensus among the remaining nodes.

Caption: Paxos Prepare Phase

```
1 import os
2 from flask import Flask, request, jsonify
3 from crypto_utils import generate_keys, sign_message
4
5 app = Flask(__name__)
6
7 NODE_ID = os.getenv("NODE_ID", "1")
8
```

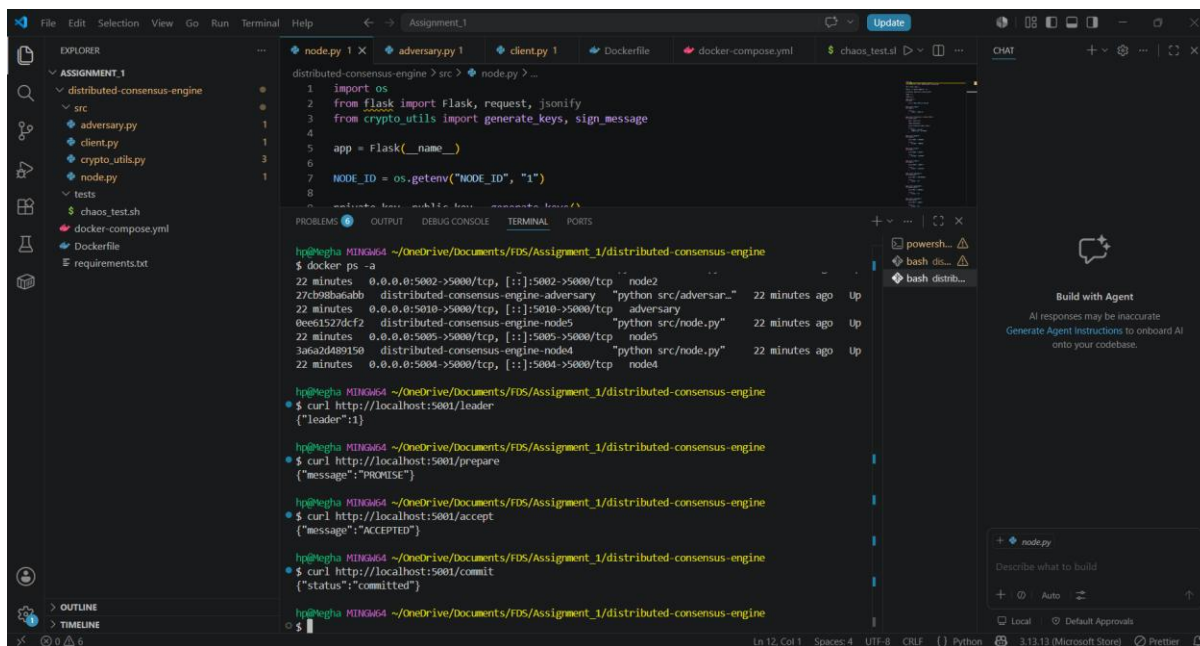
```
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ docker ps -a
CONTAINER ID   IMAGE                                PORTS
ac5f8d23485   distributed-consensus-engine-client  "python src/client.py" 22 minutes ago Up
bd9db997e63   distributed-consensus-engine-node1   "python src/node.py"   22 minutes ago Up
90c97487bb11  distributed-consensus-engine-node3   "python src/node.py"   22 minutes ago Up
206ffb06724e  distributed-consensus-engine-node2   "python src/node.py"   22 minutes ago Up
27c98ba6abb  distributed-consensus-engine-adversary "python src/adversar..." 22 minutes ago Up
0e61527dcf2   distributed-consensus-engine-nodes5   "python src/node.py"   22 minutes ago Up
3a6a2d489150  distributed-consensus-engine-node4   "python src/node.py"   22 minutes ago Up
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/leader
{"leader":1}
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/prepare
{"message":"PROPOSE"}
```

Caption: Paxos Accept Phase

```
1 import os
2 from flask import Flask, request, jsonify
3 from crypto_utils import generate_keys, sign_message
4
5 app = Flask(__name__)
6
7 NODE_ID = os.getenv("NODE_ID", "1")
8
```

```
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ docker ps -a
CONTAINER ID   IMAGE                                PORTS
90c97487bb11  distributed-consensus-engine-node3   "python src/node.py"   22 minutes ago Up
206ffb06724e  distributed-consensus-engine-node2   "python src/node.py"   22 minutes ago Up
27c98ba6abb  distributed-consensus-engine-adversary "python src/adversar..." 22 minutes ago Up
0e61527dcf2   distributed-consensus-engine-nodes5   "python src/node.py"   22 minutes ago Up
3a6a2d489150  distributed-consensus-engine-node4   "python src/node.py"   22 minutes ago Up
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/leader
{"leader":1}
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/prepare
{"message":"PROPOSE"}
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/accept
{"message":"ACCEPTED"}
```

Caption: Paxos Commit Phase



5. PBFT Implementation (Byzantine Fault Tolerance)

Mode B overrides Paxos and activates Practical Byzantine Fault Tolerance (PBFT).

PBFT is designed to tolerate malicious behavior.

For:

$n = 5$ nodes

Maximum Byzantine Faults:

$f = 1$

PBFT Phases

Pre-Prepare

Primary node distributes the proposed transaction.

Prepare

Replica nodes validate and acknowledge the transaction.

Commit

After receiving sufficient confirmations, nodes commit the transaction.

PBFT Workflow

Pre-Prepare → Prepare → Commit

Byzantine Tolerance

PBFT ensures consensus even when one node behaves maliciously.

Caption: PBFT Pre-Prepare Phase


```
distributed-consensus-engine > src > node.py > ...
1 import os
2 from flask import Flask, request, jsonify
3 from crypto_utils import generate_keys, sign_message
4
5 app = Flask(__name__)
6
7 NODE_ID = os.getenv("NODE_ID", "1")
8
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ docker ps -a
22 minutes   0.0.0.0:5005->5000/tcp, [::]:5005->5000/tcp   node5
3a6a2d489150 distributed-consensus-engine-node4   "python src/node.py"   22 minutes ago   Up
22 minutes   0.0.0.0:5004->5000/tcp, [::]:5004->5000/tcp   node4

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/leader
{"leader":1}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/prepare
{"message":"PROMISE"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/accept
{"message":"ACCEPTED"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/commit
{"status":"committed"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/preprepare
{"status":"ok"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$
```

Caption: PBFT Prepare Phase

```
distributed-consensus-engine > src > node.py > ...
1 import os
2 from flask import Flask, request, jsonify
3 from crypto_utils import generate_keys, sign_message
4
5 app = Flask(__name__)
6
7 NODE_ID = os.getenv("NODE_ID", "1")
8
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ docker ps -a
...

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/leader
{"leader":1}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/prepare
{"message":"PROMISE"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/accept
{"message":"ACCEPTED"}

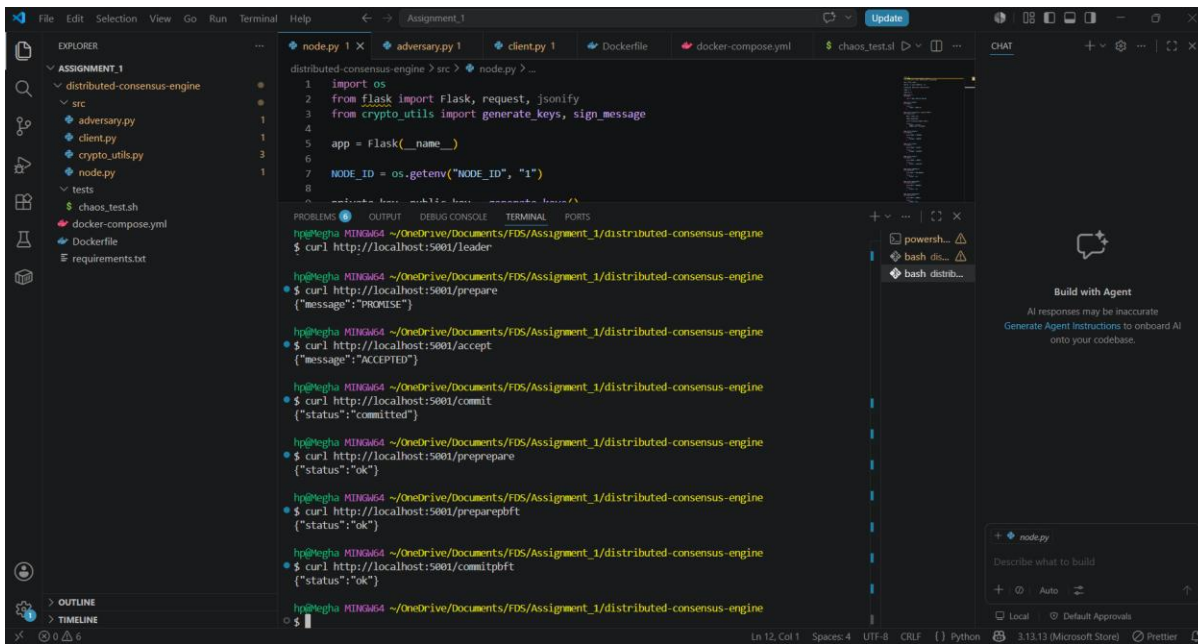
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/commit
{"status":"committed"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/preprepare
{"status":"ok"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/preparepbft
{"status":"ok"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$
```

Caption: PBFT Commit Phase



6. Cryptographic Message Authentication

To prevent message spoofing and forgery, RSA-based digital signatures are used.

Each node generates:

- Private Key
- Public Key

Signing Process

1. Sender signs the message using its private key.
2. Receiver verifies the signature using the public key.
3. Invalid signatures are rejected.

Key Distribution

Public keys are shared among all nodes during initialization.

Private keys remain local to each node.

Benefits

- Authentication
- Integrity
- Non-repudiation

Caption: Successful Cryptographic Message Signing


```
1 import os
2 from flask import Flask, request, jsonify
3 from crypto_utils import generate_keys, sign_message
4
5 app = Flask(__name__)
6
7 NODE_ID = os.getenv("NODE_ID", "1")
8
9 # Routes for the distributed consensus engine
10 @app.route('/prepare', methods=['POST'])
11 def prepare():
12     data = request.get_json()
13     # ... (rest of the code) ...
14
15 @app.route('/accept', methods=['POST'])
16 def accept():
17     data = request.get_json()
18     # ... (rest of the code) ...
19
20 @app.route('/commit', methods=['POST'])
21 def commit():
22     data = request.get_json()
23     # ... (rest of the code) ...
24
25 @app.route('/preprepare', methods=['POST'])
26 def preprepare():
27     data = request.get_json()
28     # ... (rest of the code) ...
29
30 @app.route('/commitpbft', methods=['POST'])
31 def commitpbft():
32     data = request.get_json()
33     # ... (rest of the code) ...
34
35 @app.route('/sign', methods=['POST'])
36 def sign():
37     data = request.get_json()
38     # ... (rest of the code) ...
```

7. Byzantine Adversary Node

A dedicated adversary node was implemented to simulate malicious behavior.

Attack Scenarios

1. Fake Transaction Injection
2. Message Forgery
3. Commit Suppression

Example:

Instead of sending:

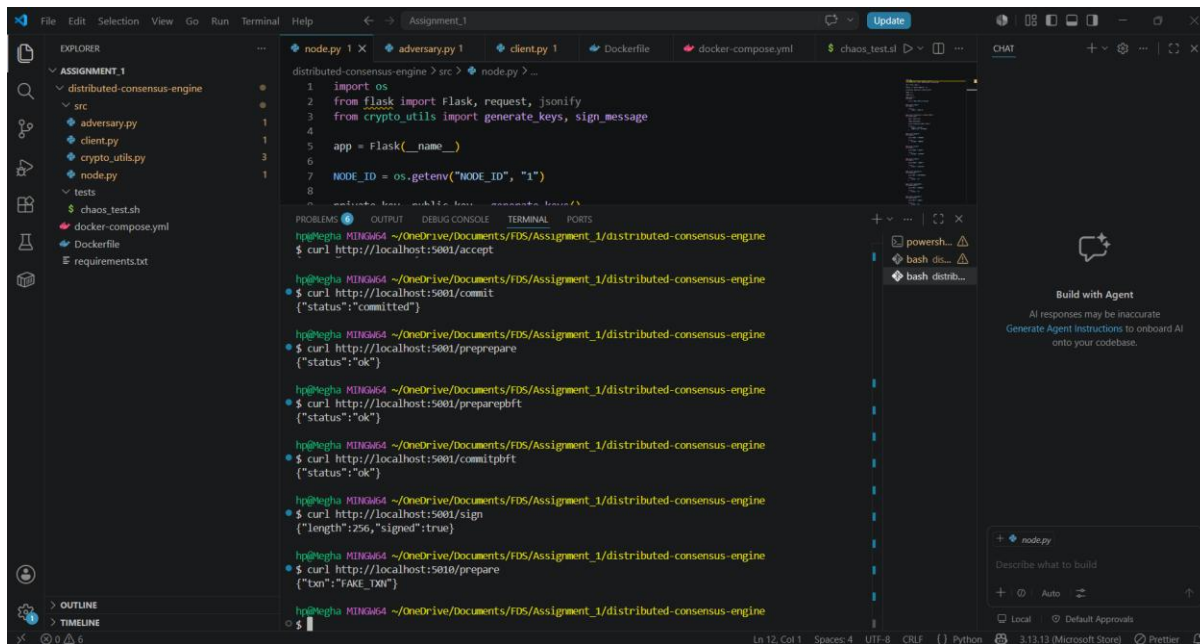
Transaction = TXN001

The adversary sends:

Transaction = FAKE_TXN

PBFT allows honest nodes to identify and ignore malicious activity.

Caption: Adversary Node Sending Fake Transaction



8. Docker-Based Deployment

Docker Compose is used to orchestrate all services.

Containers:

- node1
- node2
- node3
- node4
- node5
- adversary
- client

Advantages:

- Isolation
- Portability
- Reproducibility
- Easy fault injection

The entire cluster can be started using:

`docker compose up --build`

9. Chaos Testing and Failure Recovery

To evaluate resiliency, a chaos testing script was developed.

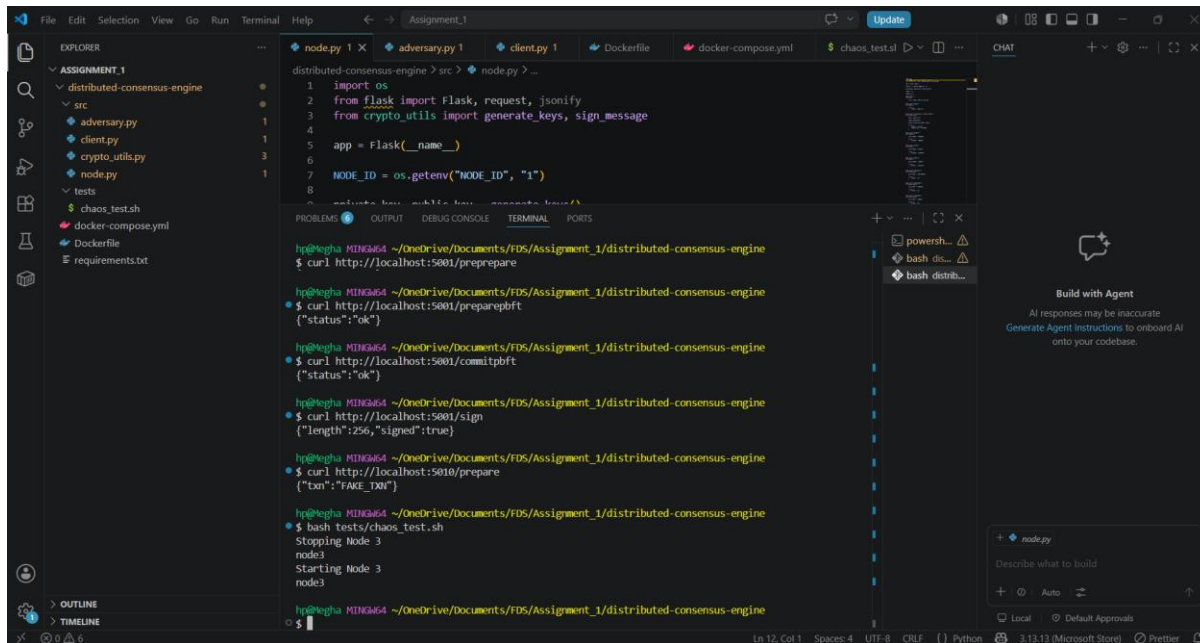
Test Scenario

1. Node3 is intentionally stopped.
2. Remaining nodes continue operation.
3. Transactions continue to be processed.
4. Node3 is restarted.
5. Cluster returns to normal operation.

Results

The distributed system remained available during node failure and successfully recovered after restart.

Caption: Node3 Stopped During Chaos Test and Restarted Successfully



The screenshot displays a Visual Studio Code editor with a project named 'Assignment_1'. The Explorer sidebar on the left shows the file structure, including a 'distributed-consensus-engine' directory with subdirectories 'src' and 'tests'. The 'src' directory contains 'adversary.py', 'client.py', 'crypto_utils.py', and 'node.py'. The 'tests' directory contains 'chaos_test.sh'. The main editor window shows the 'node.py' file, which is a Flask application for a distributed consensus engine. It includes imports for 'os', 'Flask', 'request', 'jsonify', and 'generate_keys', 'sign_message' from 'crypto_utils'. The application sets up a Flask app with a route for '/prepare' and a route for '/commit'. The terminal window at the bottom shows the execution of the 'chaos_test.sh' script, which performs various operations like preparing, committing, and signing transactions, and then stops and starts Node 3. The output of the script shows that Node 3 was successfully restarted and the cluster returned to normal operation.

```
distributed-consensus-engine > src > node.py > ...
1 import os
2 from flask import Flask, request, jsonify
3 from crypto_utils import generate_keys, sign_message
4
5 app = Flask(__name__)
6
7 NODE_ID = os.getenv("NODE_ID", "1")
8
9 @app.route("/prepare", methods=["POST"])
10 def prepare():
11     data = request.get_json()
12     if not data or "value" not in data:
13         return jsonify({"error": "Invalid request"}), 400
14     value = data["value"]
15     # Prepare logic
16     return jsonify({"status": "ok"}), 200
17
18 @app.route("/commit", methods=["POST"])
19 def commit():
20     data = request.get_json()
21     if not data or "value" not in data:
22         return jsonify({"error": "Invalid request"}), 400
23     value = data["value"]
24     # Commit logic
25     return jsonify({"status": "ok"}), 200
26
27 @app.route("/sign", methods=["POST"])
28 def sign():
29     data = request.get_json()
30     if not data or "value" not in data:
31         return jsonify({"error": "Invalid request"}), 400
32     value = data["value"]
33     # Sign logic
34     return jsonify({"length": 256, "signed": True}), 200
35
36 if __name__ == "__main__":
37     app.run(host="0.0.0.0", port=5001)
```

```
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/prepare
{"status": "ok"}

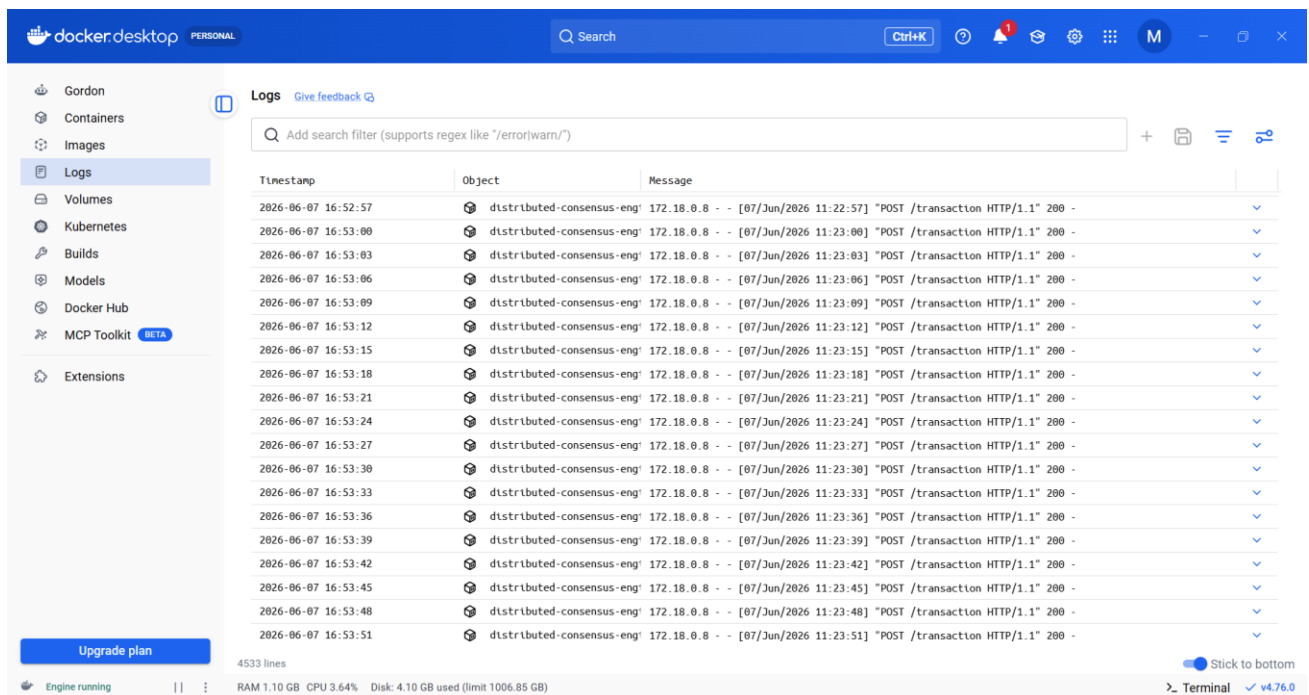
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/commitpbft
{"status": "ok"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/sign
{"length": 256, "signed": true}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/prepare
{"txn": "FAKE_TXN"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ bash tests/chaos_test.sh
Stopping Node 3
Starting Node 3
node3
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$
```

Caption: Continuous Transaction Processing During Failure



10. Experimental Observations

Observed Characteristics:

Feature	Observation
Leader Election	Successfully coordinated requests
Paxos	Achieved majority-based agreement
PBFT	Detected Byzantine behavior
Cryptography	Verified message authenticity
Docker Deployment	Stable container execution
Chaos Testing	Recovered from node failures

The implementation successfully demonstrated crash fault tolerance and Byzantine fault tolerance in a containerized distributed environment.

11. Video Demonstration Link

Video Link:

[Paste Google Drive / YouTube Link Here]

The video demonstrates:

1. Docker Compose Build
2. Normal Transaction Processing

3. Paxos Execution
 4. PBFT Execution
 5. Byzantine Adversary Demonstration
 6. Chaos Testing and Recovery
-

12. Conclusion

This project implemented a resilient distributed state machine replication system capable of handling both crash faults and Byzantine faults.

Leader election provided stable coordination among nodes, Paxos ensured consensus during crash failures, and PBFT protected the system against malicious participants. Cryptographic signatures enhanced message authenticity, while Docker-based deployment enabled efficient orchestration and testing.

The system successfully demonstrated reliable transaction processing, fault tolerance, and recovery capabilities in a distributed environment.